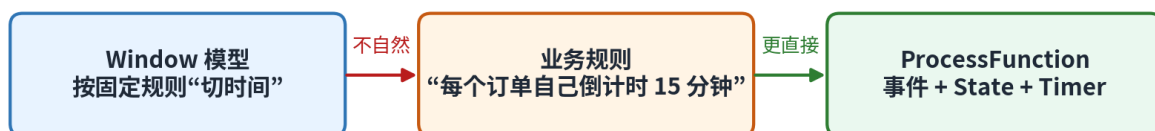


Flink 原理详解：第 8 讲

Timer、ProcessFunction 与低层流处理控制：让每个 Key 自己“定闹钟”

本讲目标：理解 ProcessFunction 为什么是 Flink DataStream API 中非常重要的低层能力，并掌握“事件 + Keyed State + Timer”的通用模式。学完后，你应该能解释并设计“订单 15 分钟未支付报警”这类事件驱动规则。

为什么有了 Window，还需要 ProcessFunction？



Window 适合“每 5 分钟统计一次”；ProcessFunction 适合“针对每个 Key，在某个未来时刻执行动作”。

核心差别：Window 是高层时间分组；ProcessFunction 是更低层的事件驱动控制。

图 1 Window 是高层时间分组；ProcessFunction 提供更自由的事件驱动控制

1. 为什么普通 Window 有时不够用？

前面我们已经学过 Window。Window 特别适合“每 5 分钟统计销售额”“每 1 小时统计访问次数”这类周期性聚合。但现实业务里还有另一类问题：每一条业务对象都有自己的时间点。

- 订单 A 在 10:00 创建，应该在 10:15 检查是否支付。
- 订单 B 在 10:07 创建，应该在 10:22 检查。
- 设备 X 30 秒没有心跳就报警。
- 用户登录失败后 10 分钟内如果连续失败 5 次就升级风险。

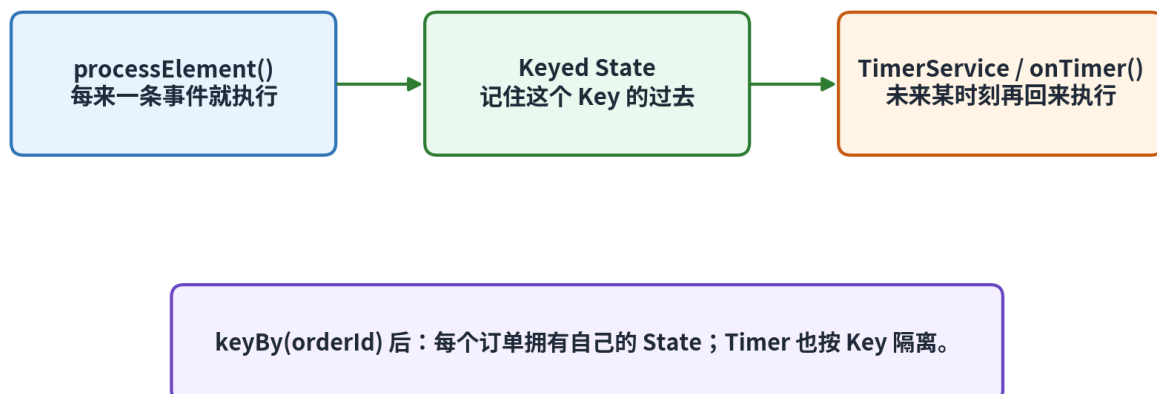
这类规则不天然属于统一的“5 分钟窗口”。它更像给每个 Key 设置一个独立的闹钟，到时间以后再回来执行逻辑。

如果问题是“统一切窗口后一起计算”，优先考虑 Window；如果问题是“每个 Key 在未来某时刻做一件事”，Timer 往往更自然。

2. ProcessFunction 到底是什么？

ProcessFunction 可以理解作为一种更低层的流处理函数。和普通 map / flatMap 相比，它不仅能处理当前事件，还能访问时间、TimerService，并在 KeyedStream 上访问 Keyed State。

KeyedProcessFunction 的三个核心能力



因此它可以实现“有状态 + 延迟触发”的自定义实时规则。

图 2 KeyedProcessFunction = processElement + Keyed State + Timer

最重要的抽象：processElement() 处理“现在来的事件”；State 记住“过去”；Timer 负责“未来再回来”。

能力	它回答的问题	典型 API / 概念
事件处理	当前这条事件来了，我现在做什么？	processElement(...)
记忆	这个 Key 之前发生过什么？	ValueState / ListState / MapState
时间	未来什么时候再执行一次？	TimerService
回调	时间到了以后做什么？	onTimer(...)
输出	是否产生结果/报警？	Collector / side output

3. 为什么重点是 KeyedProcessFunction？

ProcessFunction 家族有多个成员，但最常见、最容易形成“每个业务实体一份状态”的是 KeyedProcessFunction。它通常放在 keyBy(...) 之后。

```
stream
    .keyBy(order -> order.orderId)
    .process(new OrderTimeoutFunction());
```

当 Key 是 orderId 时，order=1001 和 order=2002 的 State 与 Timer 相互隔离。于是我们就能非常自然地表达：每个订单自己维护创建时间、支付状态和超时时刻。

Keyed State 和 Timer 都是“按 Key 作用”的。没有 keyBy，就失去了这种按业务实体隔离的语义。

4. TimerService : Flink 的“定时器管理器”

TimerService 允许函数查询当前时间、注册 Timer、删除 Timer。经典 DataStream API 中最重要的是两类 Timer：Processing-Time Timer 和 Event-Time Timer。

Processing-Time Timer vs Event-Time Timer



选择时间语义时，先问：规则依据“系统现在几点”，还是“业务事件什么时候发生”？

图 3 两种 Timer 的时间基准不同

类型	触发依据	是否依赖 Watermark	适合场景
Processing-Time Timer	机器当前处理时间	否	心跳、系统超时、重试
Event-Time Timer	事件时间进度	是	订单截止、业务时间规则

5. Processing-Time Timer : 按“系统现在几点”触发

Processing Time 使用算子所在机器的系统时钟。比如“从现在开始 30 秒后重试一次”可以直接按当前 processing time + 30 秒注册。

```
long t = ctx.timerService().currentProcessingTime() + 30_000L;
ctx.timerService().registerProcessingTimeTimer(t);
```

它不关心事件自己的时间戳，也不等待 Watermark。优点是简单、响应直接；缺点是结果会受机器调度、重启、系统负载等影响，不适合表达严格的业务事件时间语义。

6. Event-Time Timer : 按“业务事件时间”触发

Event-Time Timer 依赖 Watermark。当 Watermark 推进到某个时间戳时，对应的 Event-Time Timer 才有机会触发 onTimer()。

```
long deadline = createTs + 15 * 60 * 1000L;  
ctx.timerService().registerEventTimeTimer(deadline);
```

Event-Time Timer 的“到点”不是看墙上时钟，而是看 Watermark 是否已经证明事件时间推进到那个位置。

7. 完整案例：15 分钟未支付报警

案例：订单创建 15 分钟仍未支付就报警

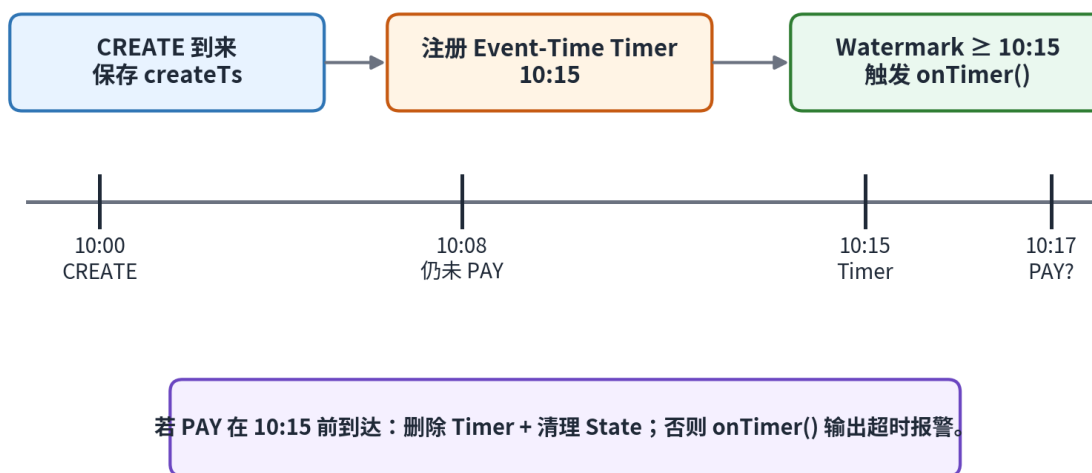


图 4 每个订单都有自己的超时时刻

先做一个简化假设：同一个订单的 CREATE 事件会先于 PAY 事件到达。后面学 CEP / 更复杂状态机时，再处理严重乱序、重复事件和补偿逻辑。

7.1 需要保存什么 State ?

最小状态可以包括：订单是否处于等待支付状态、创建时间、当前注册的 Timer 时间戳。Timer 时间戳需要保存，是因为业务提前完成时我们要准确删除 Timer。

```
private transient ValueState<Long> createTsState;  
private transient ValueState<Long> timerTsState;
```

7.2 CREATE 到来：保存状态 + 注册 Timer

```
// 伪代码 / Java API 思路
if (event.type.equals("CREATE")) {
    long createTs = event.timestamp;
    long deadline = createTs + 15 * 60 * 1000L;

    createTsState.update(createTs);
    timerTsState.update(deadline);
    ctx.timerService().registerEventTimeTimer(deadline);
}
```

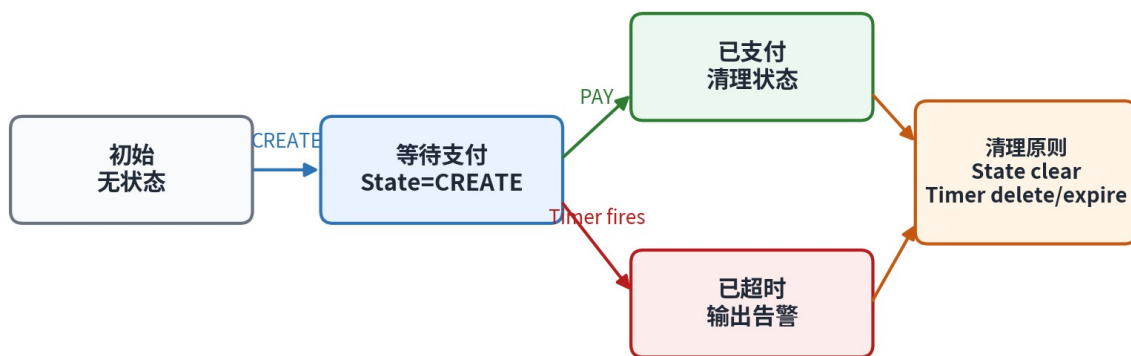
7.3 PAY 到来：取消 Timer + 清理 State

```
if (event.type.equals("PAY")) {
    Long timerTs = timerTsState.value();
    if (timerTs != null) {
        ctx.timerService().deleteEventTimeTimer(timerTs);
    }
    createTsState.clear();
    timerTsState.clear();
}
```

7.4 Timer 触发：输出超时报警

```
@Override
public void onTimer(long timestamp, OnTimerContext ctx, Collector<Alert> out) {
    Long createTs = createTsState.value();
    if (createTs != null) {
        out.collect(new Alert(ctx.getCurrentKey(), "15 分钟未支付"));
    }
    createTsState.clear();
    timerTsState.clear();
}
```

State + Timer：把业务规则写成“小状态机”



很多实时规则本质上不是“大算法”，而是：事件驱动 + 每 Key 状态机 + 定时回调。

图 5 这个订单规则可以看成是一个很小的状态机

8. 为什么 Timer 不等于 Thread.sleep()？

初学者很容易把 Timer 理解成“启动一个线程睡 15 分钟”。这是错误的。Flink Timer 是由运行时统一管理的逻辑定时器，它与 Keyed State、Checkpoint、恢复机制协同工作。

Timer 为什么能容错？——它不是普通线程 sleep()



Flink 管理的 Timer 与 Keyed State 一起参与容错恢复；它不是 JVM 里无法恢复的“睡眠线程”。

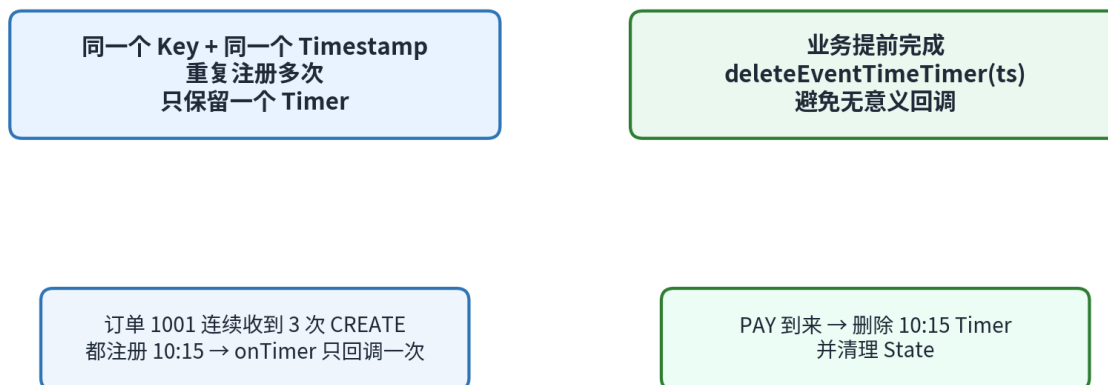
因此 ProcessFunction 可以构建长期运行、可恢复的事件驱动应用。

图 6 Timer 是 Flink 管理的可恢复运行时状态的一部分

如果你真的在 processElement() 里 Thread.sleep(15 分钟)，这个 Task 几乎等于被你锁死；它既不能高效处理其他事件，也不具备正确的分布式容错语义。

9. Timer 的两个重要工程细节

Timer 的两个重要细节：去重与取消



工程习惯：如果未来动作已经没有意义，就及时取消 Timer；如果要取消，通常需要把 timer timestamp 也存进 State。

图 7 同 Key+同 timestamp 的 Timer 会去重；无用 Timer 应及时取消

9.1 同一 Key、同一 timestamp 的 Timer 会去重

TimerService 对“同一个 Key + 同一个时间戳”的 Timer 做去重：重复注册不会导致 onTimer() 被重复调用多次。这个特性可以减少重复 Timer。

9.2 为什么要主动删除 Timer？

如果支付已经完成，10:15 的超时 Timer 就没有业务意义了。虽然 onTimer() 里也可以再次检查 State 后什么都不做，但大量无意义 Timer 会占用管理开销。因此有能力取消时，通常应该及时取消。

10. onTimer() 和 processElement() 会不会同时修改同一份 State？

Flink 会同步同一个 operator 实例中 onTimer() 与 processElement() 的调用，因此用户通常不需要自己为它们之间的 State 修改加锁。

这不等于“整个分布式系统只有一个线程”，而是说：同一个 Subtask 中这两个回调不会并发踩同一份 managed state。

11. Timer 与 Checkpoint / 故障恢复

Flink 管理的 Timer 具备容错语义。对于 KeyedProcessFunction，Timer 与对应的 Keyed State 一起参与恢复。故障后，Task 恢复 State 和 Timer，再继续根据 Event Time 或 Processing Time 语义工作。

Processing-Time Timer 有一个重要直觉：如果恢复时它的目标时间已经过去，恢复后可能很快触发。Event-Time Timer 则仍然依赖 Watermark 推进。

12. Timer 太多会不会有成本？

会。Timer 是运行时需要维护的数据结构。若你为每条事件都注册大量独立 Timer，Timer 数量可能达到百万甚至更多，影响内存/状态与 Checkpoint 成本。

- 能合并的 Timer 尽量合并到较粗粒度时间点（例如整秒、整分钟）。
- 同一个 Key + timestamp 的重复注册本身会去重，但不同 timestamp 仍是不同 Timer。
- 业务已经完成时及时 delete Timer。
- 不要把 Timer 当成“每条记录都建一个对象且永不清理”的免费资源。

原则：State 和 Timer 都应该有生命周期。长期流任务中，“只增不减”的 State/Timer 最终一定要付出代价。

13. Window 与 ProcessFunction 怎么选择？

Window 与 ProcessFunction：什么时候用谁？

问题	例子	优先工具	原因
固定时间统计	每 5 分钟销售额	Window	结构天然匹配
每 Key 独立超时	订单 15 分钟未支付	KeyedProcessFunction	每个订单独立 Timer
复杂状态机	登录失败→成功→异常	ProcessFunction / CEP	需要事件顺序与状态
任意未来回调	30 秒后重试一次	Processing-Time Timer	不依赖 Watermark
业务时间截止	活动结束后结算	Event-Time Timer	由 Watermark 推进

不要为了“高级”而用 ProcessFunction；能用高层 Window/SQL 表达的任务，通常优先用高层 API。

图 8 高层 Window 与低层 ProcessFunction 的选择

ProcessFunction 更灵活，但灵活意味着你要自己负责更多细节：State 生命周期、Timer 注册/取消、迟到事件、重复事件、恢复后的语义等。能被 Window 或 SQL 清晰表达的问题，一般没有必要为了“高级”而下降到 ProcessFunction。

14. 一个很关键的乱序问题：PAY 晚到了怎么办？

假设 CREATE 的 event time 是 10:00，deadline=10:15。Watermark 已经超过 10:15，于是 onTimer() 发出了“未支付”告警。但随后因为极端网络延迟，一个 event time=10:10 的 PAY 才到达。

这不是 Timer 错了，而是系统依据 Watermark 做出了“10:15 以前事件应该基本到齐”的判断。要处理这种业务，需要结合允许迟到策略、补偿事件、撤销告警、等待更长的 Watermark 延迟，或更复杂的状态机。

Timer 的正确性依赖你如何定义“时间已经足够确定”。Watermark 是概率/业务权衡，不是宇宙真理。

15. ProcessFunction 家族还有哪些成员？

函数	处理对象	典型用途
ProcessFunction	普通 DataStream	底层事件处理、时间访问
KeyedProcessFunction	KeyedStream	每 Key State + Timer
CoProcessFunction	两条连接流	双流自定义逻辑
KeyedCoProcessFunction	两条 keyed 连接流	双流 State + Timer
BroadcastProcessFunction	广播规则 + 普通流	动态规则下发
KeyedBroadcastProcessFunction	广播规则 + keyed 流	规则广播 + 每 Key State

这一讲先掌握 KeyedProcessFunction 即可。后续遇到“双流匹配、规则动态更新、复杂事件模式”时，再扩展到 CoProcess / Broadcast / CEP。

16. 手工模拟：你自己当一次 TimerService

事件：

10:00 order=1001 CREATE

10:03 order=2002 CREATE

10:05 order=1001 PAY

Watermark: 10:18

规则：CREATE 后 15 分钟未 PAY 则报警

请手工维护两张表：

orderId	State	Timer
1001	CREATE 后被 PAY 清理	10:15 注册后删除
2002	仍等待 PAY	10:18

Watermark 到 10:18 时：1001 不报警；2002 的 Timer 到期，onTimer() 输出报警。

17. 常见误区

误区	正确理解
Timer 就是 sleep	Timer 是 Flink 运行时管理的逻辑定时器
注册 Timer 后一定按墙钟准点触发	Event-Time Timer 依赖 Watermark；系统调度也有实际延迟
Timer 不需要 State	很多业务要取消/更新 Timer，因此常需保存 timer timestamp
Window 能做的都应该用 ProcessFunction	高层 API 能表达时优先高层 API
Timer 可以无限注册	大规模 Timer 有状态和 checkpoint 成本
Event-Time Timer 一定不会误报	Watermark 的延迟策略决定“等待多久”
processElement 与 onTimer 要自己加锁	同一 operator 实例中的调用由 Flink 同步

18. 练习题

1. 设备每 30 秒应该上报一次心跳，连续 90 秒没有心跳就报警。你更倾向 Event-Time Timer 还是 Processing-Time Timer？为什么？
2. 用户提交订单后 15 分钟未支付报警。CREATE 到来时应该保存哪些最小 State？
3. 订单支付成功后为什么要删除已注册的 Timer？
4. 如果同一个订单重复注册 3 次完全相同 timestamp 的 Timer，会触发几次 onTimer()？
5. 如果要求“每 5 分钟统计每个城市销售额”，应该优先 Window 还是 ProcessFunction？

19. 参考答案

1. 通常优先 Processing-Time Timer，因为“多久没有收到心跳”更接近系统观察时间；若业务明确定义心跳的事件时间语义，也可另行设计。
2. 至少需要“订单处于等待支付”的状态；工程上常保存 create timestamp 与 timer timestamp，方便恢复和取消。
3. 业务已完成 Timer 没有意义；删除可减少无效回调和运行时管理开销。
4. 同一个 Key + 同一个 timestamp 会被 TimerService 去重，onTimer() 只调用一次。
5. 优先 Window，因为这是标准的固定时间分组聚合问题。

20. 本讲总结

把这一讲压缩成一句话：KeyedProcessFunction 让你把“当前事件 + 过去状态 + 未来定时动作”组合起来，从而实现每个 Key 独立的事件驱动状态机。

请把下面这条主线记住：

keyBy → processElement → update State → register Timer → onTimer → emit / clear State

下一讲将进入一个非常典型的高级实时处理问题：双流匹配与复杂事件模式。课程会先从“支付流 + 订单流如何匹配”讲起，再自然引出 CEP（Complex Event Processing，复杂事件处理）的核心思想。